GH-300 GitHub Copilot

Every skill in the current study guide (skills measured as of 7 August 2026): responsible AI, Copilot in the IDE, CLI and github.com, agents and MCP, customisation, administration, data flow, prompting, testing, privacy and content exclusions. Picture first, then the facts and traps.



Pass mark 700 · Online or test centre · Renew yearly, free online · Mostly GA features

WHERE THE MARKS ARE (RANGES FROM THE STUDY GUIDE)

15–20%	25–30%	10–15%	10–15%	10–15%	10–15%
Responsible AI	Copilot features	Data & architecture	Prompting	Productivity	Privacy

Features is the biggest block: IDE, Copilot CLI, agents, MCP, code review, Spaces, Spark, customisation and organisation policies.

01	Exam blueprint	2	08	Customising Copilot	9	15	Testing & security	15
02	Surfaces & plans	3	09	Chat: commands & limits	10	16	Content exclusions & privacy	15
03	Responsible AI	4	10	Org & enterprise admin	11	17	Safeguards & troubleshooting	16
04	Copilot in the IDE	5	11	Data flow & architecture	12	18	If you see X, think Y	17
05	Agent mode, MCP, sessions	6	12	Limits of LLMs	13	19	30 self-check scenarios	18
06	Copilot CLI	7	13	Prompt engineering	13	20	Exam day	20
07	Copilot on GitHub.com	8	14	Productivity use cases	14	21	Glossary	21

THE COPILOT MAP IN ONE PICTURE

In the IDE

Inline suggestions · Next edit suggestions · Agents: Ask · Plan · Agent (Edit = legacy) · Inline chat · MCP tools · Agent sessions · Subagents

In the terminal

Copilot CLI · Interactive sessions · Programmatic -p · /model · /delegate · MCP · Custom agents

On GitHub.com

Chat · Cloud agent · Code review · PR summaries · Spaces · Spark · Mobile

Customisation

copilot-instructions.md · *.instructions.md · *.prompt.md · .github/agents/ · AGENTS.md · SKILL.md · hooks

Administration

Policies · Seats · Code review policy · Audit log · REST API · AI credits & budgets

Safeguards

Content exclusions · Public code filter · Code referencing · Proxy filters · Training opt-out

ORIENTATION

01 Exam blueprint

A practitioner exam on using Copilot well, safely and at organisation scale.

GH-300 tests whether you can use GitHub Copilot to **improve productivity, quality and security**: knowing which feature fits which job, how Copilot builds prompts and handles data, how to write good prompts, and how administrators control it with policies, content exclusions and safeguards. Expect scenario questions ("a developer wants to...", "an organisation must...") rather than trivia, plus feature and setting names.

Domain	Weight	What it tests
Use GitHub Copilot responsibly	15–20%	Risks and limitations of generative AI · ethical and responsible use · harms and mitigations · validating AI output · operating Copilot responsibly
Use GitHub Copilot features	25–30%	Copilot in the IDE (inline, chat, CLI, agent mode, content exclusions) · Copilot CLI · agent mode, Copilot Edits, MCP, agent sessions, subagents · code review · Spaces, Spark, PR summaries, instruction files · Chat limits, feedback, commands, prompt files · org policies, audit log, REST API
Copilot data and architecture	10–15%	Data usage, flow and sharing · input processing and prompt building · proxy filtering and post-processing · suggestion lifecycle · LLM limitations
Prompt engineering & context	10–15%	Prompt structure and context · how context is determined · zero-shot and few-shot · best practices · prompt process flow and chat history
Developer productivity	10–15%	Generation, refactoring, documentation · learning and less context switching · sample data and legacy modernisation · unit and integration tests · edge cases and assertions · security and performance suggestions
Privacy, exclusions, safeguards	10–15%	Content exclusions and editor settings · ownership and limits of outputs · public code filter · troubleshooting suggestions and exclusions

FORMAT AT A GLANCE

- Pass mark **700** (scaled); multiple-choice and multi-select scenario questions.
- Mostly GA features; commonly used **Preview** features may appear.
- Skills measured as of **7 August 2026**.
- Renew yearly with a **free online assessment** on Microsoft Learn.

How the best answer usually looks

Human stays accountable

Review, test and understand every suggestion. Copilot assists; the developer owns the result.

Right surface for the job

Inline for small completions, Ask for questions, agent mode for multi-file tasks, cloud agent for issues to PRs, CLI in the terminal.

Give context, not luck

Open relevant files, use #file / #codebase, write specific prompts, add instruction files for team standards.

Know the safeguard

Secrets → content exclusion (and know its gaps). Licence risk → public code filter. Policy → org settings, not each developer.

Admin at the right level

Enterprise policy overrides organisation; organisation overrides user. Seats and usage via settings or REST API.

Business data stays private

Business/Enterprise data isn't used for training; IDE prompts on those plans aren't retained.

02

READ FIRST

Copilot surfaces & plans

Where Copilot runs, who gets which features, and the June 2026 billing change.

Copilot is one product with several **surfaces**: suggestions in your editor, chat, autonomous agents in the IDE and terminal, and a cloud agent on GitHub. What you can use depends on your **plan**, and in organisations on the **policies** admins set. Many exam questions start by asking which surface or plan fits a scenario.

THREE KINDS OF HELP, FROM PASSIVE TO AUTONOMOUS

Suggestions
inline completions + next edit suggestions while you type

→

Chat
ask, explain, edit; you drive every step

→

Agents
plan, edit many files, run tools and tests (agent mode, CLI)

→

Cloud agent
works in the background from an issue and opens a PR

More autonomy means more review: the developer still approves terminal commands, reviews diffs and merges PRs.

Plan	For	Key points
Free	Individuals trying Copilot	Up to 2,000 completions a month plus an AI credit allowance; limited chat/agent use and a selection of models; not available if you already get Copilot through an organisation
Pro	Individual developers	Unlimited completions and next edit suggestions; monthly AI credits for chat, agents, CLI, cloud agent, code review
Pro+ · Max	Heavy individual users	Larger credit allowances, premium models; Max adds the highest allowance and priority access
Business	Organisations	Seats per user, pooled credits, **organisation policies**, **content exclusions**, audit log, no training on your data, IP indemnity
Enterprise	GitHub Enterprise Cloud	Everything in Business plus enterprise-level policies and larger allowances; enterprise owner chooses the plan per organisation

BILLING AND NAMING CHANGES TO KNOW

- Since **1 June 2026** usage is metered in **AI Credits** (1 credit = $0.01); older questions say **premium requests**. Same idea.
- Completions and next edit suggestions are **not** billed on paid plans.
- "Copilot coding agent" is now the **cloud agent**; both names may appear.

DOMAIN 1 · 15–20%

03 Using Copilot responsibly

Risks, harms, mitigations, and why every suggestion needs validation.

Copilot generates code that **looks right** because it predicts likely text, not because it understands your requirements. Responsible use means knowing where that fails, keeping a human accountable for every change, and putting process around AI output: review, tests, scanning and clear policies.

Risks & limitations

- Hallucinated APIs, packages and facts
- Insecure patterns copied from training data
- Outdated libraries (knowledge cutoff)
- Bias in names, comments, examples
- Code resembling public code (licensing)
- Over-reliance and skill atrophy

Potential harms

- Security vulnerabilities shipped to production
- Privacy leaks (secrets, PII in prompts or code)
- IP / licence infringement
- Unfair or exclusionary behaviour in software
- Loss of trust from unreviewed changes

Mitigations

- Human review and pull-request approval
- Tests, linters, code scanning, secret scanning
- Public code filter + code referencing
- Content exclusions for sensitive files
- Team policies and training
- Feedback (thumbs up/down) to improve

MICROSOFT'S SIX RESPONSIBLE AI PRINCIPLES

Fairness
Treat all people fairly; watch for bias.

Reliability & safety
Behave as intended, even in edge cases.

Privacy & security
Protect data; respect consent.

Inclusiveness
Empower everyone, including people with disabilities.

Transparency
People understand what the AI does and its limits.

Accountability
Humans remain responsible for outcomes.

VALIDATE BEFORE YOU TRUST

Suggestion from Copilot → **Read it** do you understand every line? → **Test it** unit, integration, edge cases → **Scan it** code scanning, secrets, dependencies → **Review it** PR approval by a human

CLASSIC WRONG ANSWERS

"Copilot's output is tested by GitHub, so it can be merged directly", "Copilot understands the business requirements", or "turning on the public code filter guarantees secure code". All false: the filter addresses matching public code, not security; validation is always the developer's job.

DOMAIN 2 · FEATURES

04 Copilot in the IDE

Enable it, trigger it, and pick the right chat mode.

Copilot runs as an extension in VS Code, Visual Studio, JetBrains IDEs, Xcode, Eclipse, Vim/Neovim and others. You sign in with a GitHub account that has a Copilot seat, then get suggestions as you type and a chat panel with several **modes**. Feature availability varies by IDE, and organisations can switch features off with policies.

Inline suggestions

Ghost text as you type or from a comment. **Tab** accept, **Esc** dismiss, **Alt/Option +]** / **[** cycle, **Ctrl+→** accept a word.

Next edit suggestions

Predicts the next place you'll need to edit (e.g. renamed variable elsewhere) and offers it.

Inline chat

Ctrl/Cmd+I in the editor: ask for a change to the selected code right there.

Chat view

Side panel conversation with participants, slash commands, context variables and model picker.

CHAT MODES IN VS CODE

Ask

Questions and explanations. Suggests code; doesn't change files on its own.

Edit · legacy

Copilot Edits: you chose the files (working set). **Deprecated in VS Code 1.110**, hidden by default; Agent now covers it.

Agent

Autonomous: finds files, edits, runs terminal commands and tools (incl. MCP), iterates on errors and tests.

Plan

Researches and produces an implementation plan first, then hands off to agent mode to build.

VS Code now calls these **agents**: built-ins are **Ask, Plan, Agent**; custom chat modes became custom agents. The study guide still says "Copilot Edits", so know both names.

ENABLING AND CONTROLLING COPILOT IN THE IDE

- Install the extension, sign in with a GitHub account that has a seat; enable/disable per language or workspace.
- **Content exclusions** apply to inline suggestions and Ask; not to Agent/Edit modes.
- Organisation policies can turn off chat, agent mode, MCP, previews or specific models.

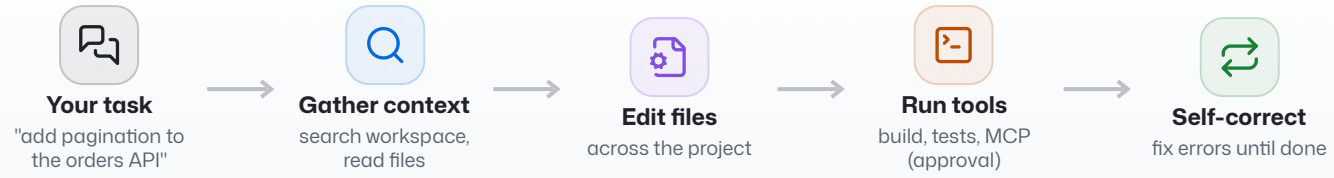
DOMAIN 2 · FEATURES

05 Agent mode, Edits, MCP & sessions

Multi-step work: tools, context management and delegation.

Agent mode moves Copilot from answering to **doing**: it finds the relevant files, makes edits across them, runs builds and tests, reads the errors and tries again. MCP servers extend what it can reach, sessions keep tasks separate, and subagents keep its working memory focused. Your job shifts to setting the goal, approving actions and reviewing the result.

THE AGENT MODE LOOP



You approve terminal commands and tool calls (or allow-list them), review the diff, then keep or undo changes.

Copilot Edits (legacy)

Controlled multi-file editing: you picked the working set and reviewed each change. Deprecated in VS Code 1.110 (removed from 1.126); Agent with diff review replaces it. Still named in the study guide.

Agent mode

Best when the task spans unknown files or needs build/test iteration. Uses tools automatically; can be extended with MCP servers.

MCP (Model Context Protocol)

Open standard to give Copilot tools and data: GitHub MCP server, databases, browsers, internal APIs. Configured per workspace or user (e.g. `.vscode/mcp.json`); governed by an organisation **MCP policy**.

Agent sessions

- Each agent task runs as a session you can view, switch, continue or stop
- Local sessions (IDE, CLI) and background/cloud sessions (cloud agent) appear together
- Start a new session for a new task to keep context focused

Subagents

- The main agent delegates a focused task (research, review, test writing) to a subagent
- The subagent works in its **own context window** and returns only the result
- Keeps the main conversation small: better quality and less token use

example .vscode/mcp.json

```
{ "servers": {
    "github":   { "type": "http",  "url": "https://api.githubcopilot.com/mcp/" },
    "postgres": { "type": "stdio", "command": "npx", "args": ["-y", "@modelcontextprotocol/server-postgres",
"postgresql://localhost/dev"] }
} }
```

WHICH TOOL FOR THE TASK?

Explain or quick question → Ask. Known files, controlled change → Edits (legacy) / Agent with diff review. Unknown scope, needs build/test loop → agent mode. Long task you don't need to watch → cloud agent (assign an issue, or `/delegate` from the CLI). Needs an external system → add an MCP server.

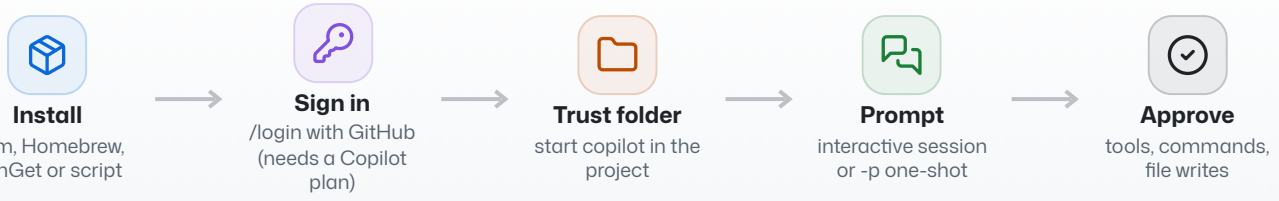
DOMAIN 2 · FEATURES

06 GitHub Copilot CLI

An agent in your terminal that understands your code and GitHub context.

GitHub Copilot CLI is a terminal-native coding agent built on the same agent harness as the cloud agent. You start it in a project folder and work conversationally: it reads files, proposes and makes changes, runs shell commands with your approval, talks to GitHub (issues, PRs) and can use MCP servers. It replaces the old `gh copilot suggest/explain` extension.

INSTALL → SIGN IN → WORK



install & use

```
npm install -g @github/copilot        # Node.js 22+
brew install --cask copilot-cli       # macOS/Linux · Windows: winget install GitHub.Copilot (PowerShell 6+)
curl -fsSL https://gh.io/copilot-install | bash                 # macOS / Linux install script
copilot                                    # interactive session in the current folder
copilot -p "Summarise open PRs that touch auth/" --allow-tool 'shell(gh)'   # programmatic, scripted use
# inside a session:
/login   /model   /delegate   /mcp   /agent   /usage   /clear   /help
@src/orders.py   reference a file in your prompt     !git status   run a shell command directly
```

/delegate hands the current task to the **cloud agent**, which opens a pull request.

CLI FACTS TO REMEMBER

- Needs a Copilot plan; an organisation or enterprise can **disable** it by policy.
- Asks before writing files or running commands; `--allow-tool` / `--deny-tool` pre-approve or block tools in scripts.
- Supports MCP, instruction files, `AGENTS.md`, custom agents. Content exclusions: **not applicable**.

07

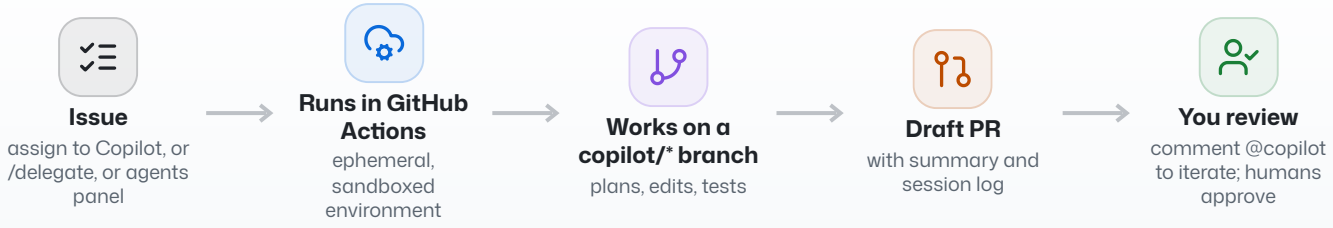
DOMAIN 2 · FEATURES

Copilot on GitHub.com

Cloud agent, code review, PR summaries, Spaces and Spark.

On GitHub.com, Copilot works where your team collaborates: it can take an issue and come back with a pull request, review other people's pull requests, write PR descriptions, answer questions over a curated set of repositories and documents, and even build small apps. Everything still flows through normal GitHub controls: branches, reviews and required approvals.

COPILOT CLOUD AGENT (FORMERLY CODING AGENT)



The agent can only push to its own branch; required reviews and branch protections still apply, and workflows on its PRs need approval.

Copilot code review

- Request Copilot as a reviewer on a PR (or in the IDE on changed code)
- Leaves comments with suggested changes you can apply
- Customise standards with **instruction files** (repo-wide or path-specific)
- Admins can enable **automatic code review** via rulesets / policies
- Excluded files aren't reviewed; reviews count toward AI credits

Pull request summaries

- Generate a PR description from the diff (Copilot icon in the PR body)
- Covers what changed, why, and files affected
- Always edit for accuracy before posting

Copilot Spaces

- Curated context collections: repos, files, PRs, issues, notes, uploads
- Ask questions grounded on that context; stays current with the repo
- Share with your team or organisation (e.g. onboarding, a service's docs)

GitHub Spark

- Build and deploy small full-stack web apps from natural language
- Hosting, data storage and AI built in; edit by prompt, visually or in code
- Creates a repository so you can keep developing normally

DOMAIN 2 · FEATURES

08 Customising Copilot

Instruction files, prompt files and custom agents make answers consistent.

Instead of repeating "use our logging library, write tests with pytest" in every prompt, you put team knowledge in **files in the repository**. Copilot reads them automatically (instructions), on demand (prompt files), or as a persona (custom agents). This is also how you set **review standards** for Copilot code review.

File	Applied	Use for
`.github/copilot-instructions.md`	Automatically, repo-wide	Coding standards, frameworks, build/test commands, review guidance
`.github/instructions/*.instructions.md`	Automatically for matching paths	Path-specific rules via `applyTo:` glob in front matter (e.g. `**/*.ts`); can exclude an agent (e.g. code review)
`AGENTS.md`	Automatically by agents	Agent instructions shared across tools (agent mode, CLI, cloud agent)
`.github/prompts/*.prompt.md`	On demand: `/name`	Reusable task prompts (e.g. `/create-endpoint`). **IDE only** (not GitHub.com or CLI)
`.github/agents/` (e.g. `reviewer.agent.md`)	When selected	Custom agents: persona, instructions, allowed tools, model (e.g. security reviewer)
`.github/skills/<name>/SKILL.md`	Automatic when relevant	Agent skills: multi-step workflows with scripts and resources
`.github/hooks/*.json`	At lifecycle events	Hooks: shell commands that always run (e.g. formatter after edits, block secrets)
Personal / organisation instructions	Per user / per org on GitHub.com	Personal preferences; organisation-wide defaults (Business/Enterprise)

.github/instructions/api.instructions.md

```
---
applyTo: "src/api/**/*.py"
---
- Use FastAPI dependency injection for DB sessions.
- Every endpoint needs a pytest test in tests/api/.
- Return errors with our ErrorResponse model.
```

KEY FACTS

- Precedence: **personal > repository > organisation** (all combined).
- Instruction files shape chat, agents and code review, not plain completions.
- Commit these files so the whole team gets the same behaviour.

09

DOMAIN 2 · FEATURES

Copilot Chat: commands, context & limits

Participants, slash commands, variables, feedback and what chat can't do.

Chat is where most interactive Copilot work happens. Three small syntaxes control it: **@participants** pick a domain expert, **/commands** run a common task, and **#variables** attach specific context. Knowing them, and knowing what chat cannot see, is a frequent source of easy marks.

Chat participants (@)

@workspace	questions about the whole project
@vscode	editor features and settings
@terminal	shell and terminal output
@github	repos, issues, PRs, web search

Slash commands (/)

/explain	explain selected code
/fix	propose a fix for problems
/tests	generate tests
/doc	add documentation comments
/new	scaffold a project
/clear	start a fresh conversation

Context variables (#)

#file	a specific file
#selection	highlighted code
#codebase	search the workspace
#editor	visible editor content
#terminalSelection	selected terminal text
#fetch	a web page (agent tool)

LIMITS, OPTIONS AND FEEDBACK

- Chat sees only its **context window**: prompt, attached context, recent history. New topic → new thread.
- Responses are non-deterministic: regenerate or rephrase.
- Thumbs up/down feedback helps GitHub improve responses.
- Usage can pause when AI credits or a budget run out.

Keyboard shortcuts worth knowing (VS Code defaults)

Action	Windows / Linux	macOS
Accept inline suggestion	Tab	Tab
Next / previous suggestion	Alt +] / Alt + [	Option +] / Option + [
Accept next word only	Ctrl + →	Cmd + →
Open inline chat	Ctrl + I	Cmd + I
Open Chat view	Ctrl + Alt + I	Ctrl + Cmd + I

DOMAIN 2 · FEATURES

10 Organisation & enterprise administration

Policies, feature availability, code review, audit log and the REST API.

For Business and Enterprise, Copilot is managed centrally. Administrators decide **who gets a seat**, **which features and models** are allowed, how **code review** runs, and keep an **audit trail** of changes. Most of this is in organisation and enterprise settings, and seat management can be automated with the REST API.

WHO DECIDES: POLICY INHERITANCE

Enterprise — sets policy or lets orgs decide

→ overrides

Organisation — policies, seats, exclusions

→ overrides

Repository — content exclusions, instructions

→

User — IDE settings within allowed limits

If the enterprise sets a policy, organisations can't change it. If it's "no policy", each organisation chooses.

Policies & features

- Enable/disable: Copilot in IDEs, Chat, CLI, cloud agent, Copilot on GitHub.com, Mobile
- MCP servers in Copilot, preview features, specific models
- Suggestions matching public code: allow / block
- Third-party agents and extensions

Code review policies

- Turn Copilot code review on for members
- Automatic review on new PRs via repository / org **rulesets**
- Standards via instruction files in each repo

Seats & access

- Grant to all members, selected teams or users
- Remove inactive seats to save cost (usage data shows last activity)
- Budgets for AI credits; users can request more

Audit log events

- Organisation / enterprise audit log records Copilot **admin activity**: seat assignments, policy and setting changes, content exclusion changes, cloud agent activity
- Filter with `action:copilot`
- Does **not** contain prompts or suggestions
- Stream to a SIEM or query via the audit log API

REST API for Copilot

GET /orgs/{org}/copilot/billing	seat breakdown & settings
GET .../copilot/billing/seats	list seats + last activity
POST .../billing/selected_users	add users (or selected_teams)
DELETE .../billing/selected_users	remove users
GET .../copilot/metrics/reports/organization-28-day/latest	usage metrics (download links). Old `.../copilot/metrics` shut down 2 Apr 2026

MEASURING ADOPTION AND VALUE

- Usage metrics (dashboard + API) show active users, acceptance, and use by IDE, language and model: find unused seats and target training.

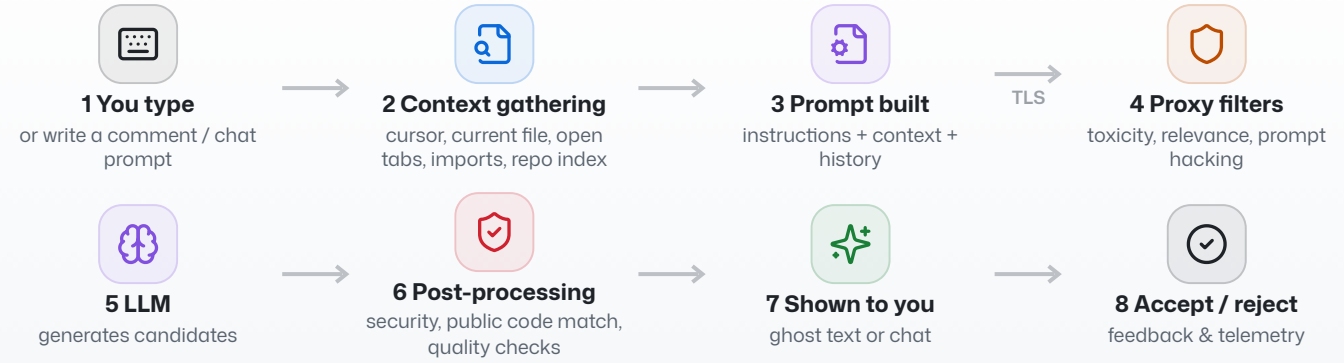
DOMAIN 3 · 10–15%

11 Data flow & architecture

From keystroke to suggestion: prompt building, proxy filters, post-processing.

Copilot never sends "your whole repo" to a model. The IDE extension **builds a prompt** from the most relevant context it can find, sends it over TLS to GitHub's **proxy**, which filters it, forwards it to the model, then **post-processes** the response before you see it. Knowing each stage answers most architecture and privacy questions.

THE SUGGESTION LIFECYCLE



How context is chosen

- Code before and after the cursor (fill-in-the-middle)
- Current file language, path, imports
- **Open tabs** (neighbouring files)
- Chat: selection, #-variables, history, instruction files
- Repository index for #codebase / @workspace

Proxy filtering

- Blocks toxic, hateful or off-topic prompts
- Detects prompt injection / jailbreak attempts
- Enforces content exclusions and policies
- Hosted by GitHub in Azure

Post-processing

- **Security filter**: blocks insecure patterns (hard-coded credentials, SQL injection)
- **Public code filter**: drops ~150-character matches to public code when blocked
- **Code referencing**: shows matches + licences when allowed

DATA USAGE AND RETENTION

- **Business & Enterprise**: data is not used to train models; prompts and suggestions from IDE completions and chat are not retained; other surfaces keep them about 28 days (some GitHub.com chat/agent history now kept longer); engagement data up to 2 years.
- **Free, Pro, Pro+ (and Max)**: since 24 April 2026 interaction data may be used for model training **unless the user opts out** in Copilot settings (Privacy).
- Private repository content **at rest** is not used for training; code is processed only while you use Copilot.
- Data is encrypted in transit (TLS) and at rest; check the GitHub Trust Center and the plan's data terms for the latest numbers.

12

DOMAIN 3

Limitations of LLMs & Copilot

Why suggestions go wrong, and what that implies.

Every Copilot limitation comes from how large language models work: they predict plausible text from patterns in training data and from the limited context they are given. They do not run your code, know your requirements or check facts. The table maps each limitation to what you should do about it.

Limitation	What happens	Implication
Probabilistic output	Predicts likely tokens; same prompt can give different answers	Review and test everything; regenerate or rephrase
Limited context window	Only part of the project is visible; long chats forget	Open/attach relevant files, start new chats per task
Knowledge cutoff	May suggest deprecated APIs or old library versions	Check docs and versions; give current examples
Hallucination	Invents functions, packages, parameters	Verify dependencies exist (also a supply-chain risk)
Training-data bias	More fluent in popular languages and patterns; may echo insecure or biased code	Extra scrutiny for niche stacks and security-sensitive code
No real understanding	Doesn't know business rules or intent	Specify requirements explicitly; tests encode intent
Public code similarity	Rarely reproduces public snippets	Public code filter or code referencing

13

DOMAIN 4 · 10–15%

Prompt engineering & context crafting

Clear asks plus the right context beat clever wording.

Copilot can only work with what it is given. A good prompt states the **goal**, supplies the **context**, sets **constraints**, shows **examples** when the format matters, and says what **output** you want. For inline suggestions, your comments, names and open files are the prompt.

ANATOMY OF A STRONG COPILOT PROMPT

Write a Python function that validates an email address **GOAL** for our signup form in #file:forms.py **CONTEXT** using only the standard library, max 30 lines **CONSTRAINTS** e.g. "a@b.co" → True, "a@b" → False **EXAMPLES** Return the function and three pytest tests. **OUTPUT**

Zero-shot

Instruction only: "Write a function to parse ISO dates." Good for common tasks Copilot already knows.

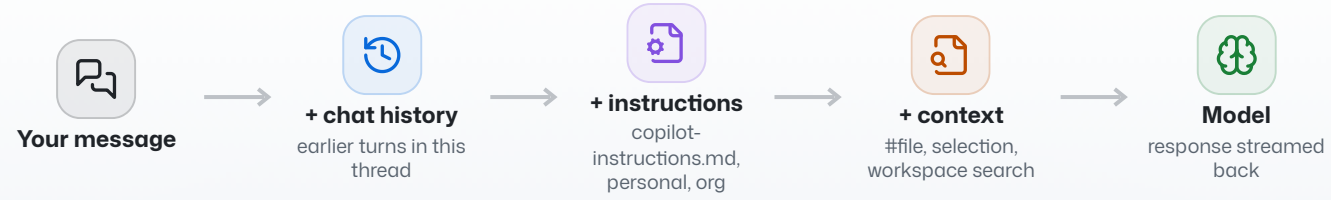
Few-shot

Instruction + 1–3 examples of input → output (or an existing function to imitate). Best for your own formats, naming and patterns.

THE 4 S'S AND OTHER BEST PRACTICES

- **Single** task · **Specific** details · **Short** wording · **Surround** with context (open files, good names, comments).
- Break big tasks into steps and iterate.
- Reference files with `#file` ; close unrelated tabs.

PROMPT PROCESS FLOW IN CHAT



14

DOMAIN 5 · 10–15%

Developer productivity

Generation, refactoring, docs, learning, sample data and modernisation.

The productivity domain is about knowing **which everyday tasks** Copilot speeds up and how to use it for each one without lowering quality. The common thread: let Copilot draft, keep tests and review as the safety net, and stay in your editor instead of switching to search engines and docs.

Generate code

Boilerplate, CRUD, regex, SQL, config files, shell scripts from comments or chat; agent mode for whole features.

Refactor

Extract functions, simplify logic, rename consistently, apply patterns; keep tests green between steps.

Document

`/doc` for docstrings, README sections, API docs, commit messages and PR summaries.

Learn faster

`/explain` unfamiliar code, ask about a new language or framework, without leaving the editor (less context switching).

Sample data

Realistic fake JSON, CSV, SQL inserts and fixtures for tests and demos; never paste real customer data.

Modernise legacy code

Explain COBOL/old Java, translate to modern languages, upgrade frameworks; generate tests first to lock behaviour.

PRODUCTIVITY PATTERN

Explain → write tests that capture current behaviour → refactor or translate in small steps with Copilot → run tests after each step → review the diff. This is the safest way to use Copilot on legacy code.

DOMAIN 5

15 Testing & security

Unit and integration tests, edge cases, assertions, safer and faster code.

Copilot is good at writing the tests developers tend to skip and at spotting common security and performance problems. It is not a substitute for test design or security tooling: you still decide what correct behaviour is, check that tests fail for the right reasons, and run scanners in CI.

Generating tests

- `/tests` on a selection or file; agent mode to add a test suite and run it
- Specify the framework (pytest, JUnit, Jest) and style
- **Unit** tests isolate one function (mocks for dependencies); **integration** tests exercise components together (DB, API)
- `/fixTestFailure` or paste the failure to fix broken tests

Edge cases & assertions

- Ask: "list edge cases for this function" before writing tests
- Empty, null, boundary values, very large input, unicode, concurrency, error paths
- Meaningful assertions on outputs and side effects, not just "no exception"
- Review tests: Copilot may write tests that pass for the wrong reason

Security improvements

- Ask Copilot to review code for OWASP issues: injection, XSS, insecure deserialisation, secrets
- Parameterised queries, input validation, safe crypto libraries
- Pair with GitHub code scanning, secret scanning, Dependabot; Copilot Autofix suggests fixes for alerts

Performance

- Ask for complexity analysis and faster algorithms or data structures
- Caching, batching, avoiding N+1 queries
- Always benchmark before and after: suggestions aren't proof

DOMAIN 6 · 10–15%

16 Content exclusions & privacy settings

Keep sensitive files out of Copilot, and know where exclusions don't apply.

Content exclusions tell Copilot to ignore specific files or repositories, typically secrets, keys, regulated data or proprietary algorithms. They are configured by admins at repository, organisation or enterprise level, only on Business and Enterprise plans, and they do **not** cover every Copilot feature.

WHERE CONTENT EXCLUSIONS ARE CONFIGURED

Repository

Settings → Copilot → Content exclusion. Paths in that repo, e.g. `- "/secrets/**"`, `- "*.env"`. Repo admins.

Organisation

Org settings → Copilot → Content exclusion. Rules for any repo (by URL or `"*"` for all) and paths. Org owners.

Enterprise

Enterprise settings: rules applied across all organisations. Enterprise owners.

Available on **Copilot Business and Enterprise**. Changes can take up to ~30 minutes to reach IDEs (reload the window to pick them up sooner).

organisation content exclusion example (YAML)

```yaml
"*":                          # every repository
  - "**/.env"
  - "**/secrets/**"
https://github.com/acme/payments:
  - "/config/keys/**"
  - "*.pem"
```

When a file is excluded

- No inline suggestions in that file
- Its content isn't used to build suggestions in other files
- Chat (Ask) can't read it; it isn't listed as a reference
- Copilot code review skips it

but

Known gaps

- Not supported in **Agent and Edit modes**; "not applicable" to **Copilot CLI**
- Not guaranteed for third-party **MCP servers**
- Doesn't apply to symlinks or remote filesystems
- The IDE may still pass semantic info (type names, hovers)
- GitHub.com / Mobile support is public preview
- Best protection: don't keep secrets in repos at all

EDITOR SETTINGS AND OWNERSHIP OF OUTPUT

- GitHub claims **no ownership** of suggestions; you are responsible for licensing and security.
- Business and Enterprise include **IP indemnity** (conditions apply).
- Individuals choose the public-code setting (if allowed) and the training opt-out.

DOMAIN 6

17 Safeguards & troubleshooting

Public code filter, code referencing, and fixing common problems.

Two safeguards deal with code that matches public repositories: the **public code filter** hides such suggestions, while **code referencing** shows them together with where they come from and their licence. Troubleshooting questions usually come down to sign-in, seats, policies, exclusions, filters or the network.

Public code: Blocked

duplication detection filter on

Suggestions are checked against public GitHub code; matches of about 150 characters or more are dropped. Safest for licence risk; may hide some useful suggestions.

or

Public code: Allowed

with code referencing

Matching suggestions are shown, and **code referencing** lists where the code appears and its licence (in the IDE log / references panel) so you can decide.

Set by the individual on personal plans, or enforced by organisation / enterprise policy on Business and Enterprise.

Problem	Check / fix
No suggestions at all	Signed in with the right account? Seat assigned? Extension enabled for this language? Network / proxy / firewall allows Copilot endpoints? Status page outage?
Suggestions stop in one file	File matches a content exclusion (look for the exclusion notice); file too large; language disabled

Problem	Check / fix
Exclusion not taking effect	Wait up to ~30 min or reload the IDE; check the path syntax and level (repo vs org); remember agent mode / CLI / cloud agent don't honour it
Suggestions vanish mid-typing	Public code filter blocked a match, or the security filter removed it
Poor or irrelevant suggestions	Add context: open related files, better names, comments; be specific in chat; add instruction files
Feature missing (chat, CLI, MCP, agent)	Disabled by organisation or enterprise policy; preview features off; plan doesn't include it
Usage paused	AI credit allowance or budget exhausted, or rate limit; buy more / request budget / wait
Corporate network issues	Configure HTTP proxy and custom certificates in the IDE; allowlist GitHub Copilot hostnames

CRAM SHEET

18 If you see X, think Y

Pattern-match in the final hour.

Responsible AI

Copilot output looks right but untested	**Human review + tests**
Invented package name	**Hallucination: verify dependency**
Who is accountable for merged code?	**The developer / team**
Principle: people understand AI limits	**Transparency**
Principle: humans answerable	**Accountability**
Reduce licence risk	**Public code filter / code referencing**
Reduce shipped vulnerabilities	**Review + code scanning**

IDE & chat

Accept suggestion	**Tab**
See alternative suggestions	**Alt/Option +] / [**
Edit code in place by prompt	**Inline chat (Ctrl/Cmd+I)**
Question, no file changes	**Ask mode**
Chosen files, controlled multi-file edit	**Copilot Edits (legacy) / Agent**
Unknown scope, run tests, iterate	**Agent mode**
Plan before building	**Plan mode**
Question about whole project	**@workspace / #codebase**
Explain terminal error	**@terminal**
Generate tests for selection	**/tests**
Fresh conversation	**/clear**

Agents, CLI & GitHub.com

Connect Copilot to a database or API	**MCP server**
Keep main chat context small	**Subagent**
AI agent in the terminal	**Copilot CLI**
Install CLI	**npm i -g @github/copilot**
Switch model in CLI	**/model**
Hand CLI task to the cloud	**/delegate**
Issue → PR in the background	**Cloud agent (coding agent)**
AI reviewer on a PR	**Copilot code review**
Auto-review every PR	**Ruleset / code review policy**
Write PR description	**PR summary**
Shared curated context for a team	**Copilot Spaces**
Build a small web app from a prompt	**GitHub Spark**

Customisation & prompting

Repo-wide coding standards	**copilot-instructions.md**
Rules only for *.ts files	***.instructions.md + applyTo**
Reusable slash prompt	***.prompt.md**
Custom persona with tool limits	**Custom agent (.github/agents/)**
Instructions shared across agents	**AGENTS.md**
No examples given	**Zero-shot**
Examples given	**Few-shot**
Improve weak suggestions	**More context, specific prompt**
Long chat drifting	**New thread**

Administration & data

Enterprise vs org policy conflict	**Enterprise wins**
Who changed Copilot settings?	**Audit log (action:copilot)**
Add seats by script	**REST API selected_users**
Find inactive seats	**Seats API last_activity_at**
Block CLI for everyone	**Org / enterprise policy**
Business data used for training?	**No**
Pro user doesn't want training	**Opt out in settings**
Where are prompts filtered?	**GitHub proxy**
Insecure pattern removed	**Post-processing security filter**

Privacy & safeguards

Keep .env out of Copilot	**Content exclusion**
Exclusions for all repos	**Org-level rule with "*"**
Which plans have exclusions?	**Business & Enterprise**
Exclusion ignored by agent	**Known limitation (agent/CLI/cloud)**
Exclusion just added, still suggesting	**Wait ~30 min / reload IDE**
Block matching public code	**Public code filter: Blocked**
See licence of matching code	**Code referencing**
Who owns accepted code?	**You (GitHub claims no ownership)**

19

PRACTICE

30 self-check scenarios

Cover the green line, answer, then check. Colour = domain.

■ Responsible AI ■ Features ■ Prompting & customisation ■ Data & architecture ■ Productivity ■ Administration ■ Privacy & safeguards

1. A junior wants to merge Copilot-generated auth code without tests because 'Copilot is trained on good code'.
→ Reject: review, test and scan it; the developer is accountable

2. Copilot suggests importing a package nobody has heard of.
→ Verify it exists and is trustworthy (hallucinated or typosquatted packages)

3. Legal worries Copilot may reproduce licensed public code.
→ Enable public code filter (Blocked) or code referencing

4. Developer wants to tweak one function without leaving the editor.
→ Inline chat (Ctrl/Cmd+I)

5. Rename a field across four known files with review of each change.
→ Copilot Edits with those files in the working set
(legacy name; today Agent with diff review)

6. Add a feature across unknown files and fix failing tests until green.
→ Agent mode

7. Agent needs to query the team's Postgres database.
→ Add an MCP server

8. Developer lives in the terminal and wants an agent there.
→ GitHub Copilot CLI

9. Hand a backlog issue to Copilot and get a PR later.
→ Assign the issue to the cloud agent

10. From the CLI, continue a long task in the background on GitHub.
→ /delegate

11. Every PR in the org should get an AI review automatically.
→ Enable Copilot code review via ruleset/policy

12. New hires need answers grounded on three repos and the wiki.
→ Copilot Spaces

13. Team wants Copilot to always use pytest and their logger.
→ copilot-instructions.md

14. Rules should apply only to Terraform files.
→ *.instructions.md with applyTo: "**/*.tf"

15. Same 'create endpoint' prompt typed weekly.
→ Prompt file (*.prompt.md) invoked with /name

16. A security-review persona limited to read-only tools.
→ Custom agent in .github/agents/

17. Suggestions ignore the helper module.
→ Open the helper file / reference it with #file

18. Where are toxic or jailbreak prompts blocked?
→ GitHub's proxy before the model

19. Does Copilot Business keep IDE chat prompts?
→ No, not retained; not used for training

20. Pro user doesn't want their prompts used for training.
→ Opt out in Copilot settings (Privacy)

21. Copilot suggests a deprecated API.
→ Knowledge cutoff: check current docs, give context

22. Need tests for a parser's tricky inputs.
→ Ask for edge cases, then /tests with specific assertions

23. Translate a legacy module safely.
→ Generate tests first, then translate in small steps

24. Code scanning found SQL injection.
→ Copilot Autofix / ask Copilot for parameterised queries

25. Enterprise disabled CLI but an org owner wants it on.
→ Not possible: enterprise policy overrides

26. Add 200 users to Copilot from HR data.
→ REST API: POST /orgs/{org}/copilot/billing/selected_users

27. Keep secrets/ out of Copilot in every repo.**
→ Organisation content exclusion with "**"

28. Exclusion added but suggestions still appear.
→ Wait ~30 min or reload the IDE; check path

29. Agent mode read an excluded file.
→ Known limitation: exclusions don't cover agent mode/CLI/cloud agent

30. Suggestions disappear on certain snippets.
→ Public code filter blocked a match

FINISH

20 Exam day

Calm, methodical, and precise about feature names.

Method

Read the last line first → identify the domain (feature, data, privacy, admin) → eliminate answers that skip human review or use the wrong level (user vs org vs enterprise) → pick the most specific feature.

Classic traps

"Copilot output is verified" · content exclusions on Free/Pro · exclusions covering agent mode · public code filter = security · Business data used for training · user overriding enterprise policy · prompt file vs instruction file.

Logistics

Try the exam sandbox first · ID ready · online: clean desk and system check · English not your first language? You can request 30 extra minutes if your language isn't offered.

Final-hours checklist

- ☐ Six domains and their weights
- ☐ Risks, mitigations, six responsible AI principles
- ☐ Plans; AI credits = old premium requests
- ☐ Ask · Plan · Agent (+ legacy Copilot Edits)
- ☐ Agent loop, MCP, sessions, subagents
- ☐ Copilot CLI: install, /login, /model, /delegate
- ☐ Cloud agent, code review, PR summaries, Spaces, Spark
- ☐ Instruction files, applyTo, AGENTS.md, prompt files, custom agents
- ☐ @participants, /commands, #variables
- ☐ Policy order enterprise › org › user; audit log; seat API
- ☐ Suggestion lifecycle; proxy vs post-processing
- ☐ Retention and training by plan
- ☐ LLM limitations; 4 S's; zero- vs few-shot
- ☐ Productivity, tests, edge cases, security
- ☐ Content exclusions: levels, timing, gaps
- ☐ Public code filter vs code referencing; ownership
- ☐ Troubleshooting table
- ☐ Re-read §18, redo §19

ONE LINE TO REMEMBER

Copilot proposes, **you decide**: give it clear prompts and the right context, pick the right surface for the task, keep sensitive content out with exclusions (knowing their gaps), filter public code when licence matters, and govern it centrally with policies.

21

REFERENCE

Glossary

Blank on a term mid-question? It's here.

Agent mode: chat mode that plans, edits files and runs tools autonomously
AGENTS.md: shared instruction file for coding agents
AI credits: usage unit since June 2026 (1 credit = $0.01)
Audit log: record of admin events, not prompts
Cloud agent: background agent on GitHub (formerly coding agent)
Code referencing: shows matching public code and its licence
Content exclusion: rule that hides paths from Copilot
Copilot CLI: terminal agent (`copilot`)
Agent skill: `SKILL.md` workflow loaded when relevant
Copilot Edits: legacy controlled multi-file editing (deprecated in VS Code)
Custom agent: persona file in `.github/agents/`
Hook: command run at agent lifecycle events
FIM: fill-in-the-middle, code before and after cursor
Instruction file: automatic context (`copilot-instructions.md`)
IP indemnity: GitHub's legal protection for Business/Enterprise
MCP: Model Context Protocol for tools and data
NES: next edit suggestions
Plan mode: research and plan before building
Policy: org/enterprise setting controlling features
Post-processing: filters applied to model output
Premium request: older usage unit before AI credits
Prompt file: reusable prompt `*.prompt.md`
Proxy: GitHub service filtering prompts and responses
Public code filter: blocks suggestions matching public code
Subagent: delegated helper with its own context

Technically validated against GitHub Docs in November 2026 (Edit mode deprecation, CLI install commands, metrics API shutdown, content-exclusion gaps, prompt-file surfaces, skills and hooks). Built against the Microsoft Learn GH-300 study guide (skills measured as of 7 August 2026) and GitHub documentation as of late October 2026. Copilot changes fast (plans moved to AI credits in June 2026; the coding agent is now the cloud agent; Copilot CLI and content exclusion behaviour are evolving), so confirm details in GitHub Docs. Diagrams are original illustrations; icons from Lucide (ISC). Not affiliated with GitHub or Microsoft.